



**Ecole nationale des
Sciences Appliquées – Fès**



Compte Rendu:

TP1 : KNN

Réaliser par :

❖ Abir Majdi

Encadré par :

Prof. AKKAD

Année universitaire : 2025/2026

Filière : GDNC2

Introduction

Dans le domaine du **Machine Learning**, les algorithmes de classification permettent de prédire la classe d'un individu à partir de ses caractéristiques. Parmi ces méthodes, l'algorithme **K-Nearest Neighbors (KNN)** est l'un des plus simples et des plus intuitifs. Il repose sur l'idée que les observations similaires se trouvent généralement proches les unes des autres dans l'espace des données.

L'algorithme KNN est une méthode **d'apprentissage supervisé** qui ne nécessite pas de phase d'entraînement complexe. Pour effectuer une prédiction, il se base sur les **k voisins les plus proches** d'un point dans le jeu de données d'entraînement et attribue la classe majoritaire parmi ces voisins.

Ce TP vise à étudier le fonctionnement de cet algorithme en l'appliquant à différents jeux de données et en analysant l'impact de plusieurs paramètres tels que la valeur de **k**, la normalisation des données et le choix de la **mesure de distance**.

Au cours de ce travail pratique, nous avons implémenté l'algorithme KNN, testé ses performances sur plusieurs jeux de données et comparé ses résultats avec d'autres modèles de classification. Des visualisations graphiques ont également été réalisées afin de mieux comprendre le comportement de l'algorithme.

Objectifs du TP

Les principaux objectifs de ce travail pratique sont les suivants :

- Comprendre le fonctionnement de l'algorithme **K-Nearest Neighbors (KNN)**.
- Implémenter l'algorithme KNN **à partir de zéro sans utiliser scikit-learn**.
- Appliquer l'algorithme sur différents jeux de données tels que **Iris, Wine et Breast Cancer**.
- Étudier l'impact de la **valeur du paramètre k** sur la précision du modèle.
- Comparer différentes **mesures de distance** utilisées par l'algorithme.
- Visualiser les **voisins les plus proches et les frontières de décision**.
- Comparer les performances du KNN avec d'autres modèles de classification.
- Analyser l'impact de différents facteurs tels que **la taille du dataset, le bruit et la normalisation** sur les performances du modèle.

Partie 1

Importation des bibliothèques

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

- pandas : manipuler les données sous forme de tableau (DataFrame)
- numpy : calcul scientifique et manipulation de tableaux
- matplotlib : visualisation graphique des données

Chargement du dataset

Code python

```
df_iris = pd.read_csv('iris.csv')
print(df_iris.tail())
```

Output

```
PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u
"c:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN\tp2.py"
   Id  SepalLength[cm]  ...  PetalWidth[cm]  Species
145  146              6.7  ...              2.3  Iris-virginica
146  147              6.3  ...              1.9  Iris-virginica
147  148              6.5  ...              2.0  Iris-virginica
148  149              6.2  ...              2.3  Iris-virginica
149  150              5.9  ...              1.8  Iris-virginica

[5 rows x 6 columns]
```

Extraction des caractéristiques (features) et des classes (labels)

Code python

```
X = df_iris[['PetalLength[cm]', 'PetalWidth[cm]']].values
X[:5, :]
print("Extraction des caractéristiques",X)
```

Output

```

PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\
IA mercredi\TP 2 KNN\TP 2 KNN\tp2.py"
Extraction des caractéristiques [[1.4 0.2]
[1.4 0.2]
[1.3 0.2]
[1.5 0.2]
[1.4 0.2]
[1.7 0.4]
[1.4 0.3]
[1.5 0.2]
[1.4 0.2]
[1.5 0.1]
[1.5 0.2]
[1.6 0.2]
[1.4 0.1]
[1.1 0.1]

```

On sélectionne deux variables du dataset :

- longueur du pétale
- largeur du pétale

Ces valeurs sont stockées dans un tableau **NumPy** appelé X.

```

IA mercredi\TP 2 KNN\TP 2 KNN\tp2.py
PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\
IA mercredi\TP 2 KNN\TP 2 KNN\tp2.py"
Extraction des classes
   Id  SepalLength[cm]  SepalWidth[cm]  ...  PetalWidth[cm]  S
pecies ClassLabel
145  146              6.7            3.0  ...            2.3  Iris-virginica    2
146  147              6.3            2.5  ...            1.9  Iris-virginica    2
147  148              6.5            3.0  ...            2.0  Iris-virginica    2
148  149              6.2            3.4  ...            2.3  Iris-virginica    2
149  150              5.9            3.0  ...            1.8  Iris-virginica    2

[5 rows x 7 columns]

```

contient les **classes des fleurs** :

- Setosa
- Versicolor
- Virginica

Ces classes sont ce que l'algorithme devra **prédire**.

```

y = df_iris['ClassLabel'].values
print(y[:5])

```

```

[0 0 0 0 0]

```

4 - Shuffle Dataset and Create Training and Test Subsets

```
indices = np.arange(y.shape[0])
print(indices)
```

```
PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\
IA mercredi\TP 2 KNN\TP 2 KNN\tp2.py"
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17
 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35
 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53
 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71
 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89
 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107
108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125
126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143
144 145 146 147 148 149]
```

```
rnd = np.random.RandomState(123)
shuffled_indices = rnd.permutation(indices)
print(shuffled_indices)
```

```
PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\
IA mercredi\TP 2 KNN\TP 2 KNN\tp2.py"
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17
 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35
 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53
 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71
 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89
 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107
108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125
126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143
144 145 146 147 148 149]
[ 72 112 132  88  37 138  87  42   8  90 141  33  59 116 135 104  36  13
 63  45  28 133  24 127  46  20  31 121 117   4 130 119  29   0  62  93
131   5  16  82  60  35 143 145 142 114 136  53  19  38 110  23   9  86
 91  89  79 101  65 115  41 124  95  21  11 103  74 122 118  44  51  81
149 12 129  56  50  25 128 146  43   1  71  54 100  14   6  80  26  70
139 30 108  15  18  77  22  10  58 107  75  64  69   3  40  76 134  34
 27 94  85 97 102  52  92  99 105   7  48  61 120 137 125 147  39  84
   2  67  55  49  68 140  78 144 111  32  73  47 148 113  96  57 123 106
 83 17  98  66 126 109]
```

```
X_shuffled, y_shuffled = X[shuffled_indices], y[shuffled_indices]
X_train, y_train = X_shuffled[:100], y_shuffled[:100]
X_test, y_test = X_shuffled[100:], y_shuffled[100:]
print(X_train, y_train, X_test, y_test)
```

```
PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\
IA mercredi\TP 2 KNN\TP 2 KNN\tp2.py"
X_train [[4.9 1.5]
 [5.5 2.1]
 [5.6 2.2]
 [4.1 1.3]
 [1.5 0.1]
 [4.8 1.8]
 [4.4 1.3]
```

```

y_train [1 2 2 1 0 2 1 0 0 1 2 0 1 2 2 2 0 0 1 0 0 2 0 2 0 0 0 2 2 0 2 2 0 0 1 1 2
 0 0 1 1 0 2 2 2 2 2 1 0 0 2 0 0 1 1 1 1 2 1 2 0 2 1 0 0 2 1 2 2 0 1 1 2 0
 2 1 1 0 2 2 0 0 1 1 2 0 0 1 0 1 2 0 2 0 0 1 0 0 1 2]
X_test [[4.4 1.4]
 [3.6 1.3]
 [3.9 1.1]
 [1.5 0.2]
 [1.3 0.3]
 [4.8 1.4]
 [5.6 1.4]
 [1.5 0.1]

```

```

y_test [1 1 1 0 0 1 2 0 0 1 1 1 2 1 1 1 2 0 0 1 2 2 2 2 0 1 0 1 1 0 1 2 1 2 2 0 1
 0 2 2 1 1 2 2 1 0 1 1 2 2]

```

5 - Doing Steps 1-4 in Scikit-Learn

```

iris = load_iris()
X, y = iris.data[:, 2:], iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.3,
                                                    random_state=123,
                                                    shuffle=True)

print("X_train",X_train)
print("y_train",y_train)
print("X_test",X_test)
print(["y_test",y_test])

```

```

PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\
IA mercredi\TP 2 KNN\TP 2 KNN\tp2.py"

```

```

X_train [[5.1 2.4]
 [5.6 2.4]
 [4.  1.3]
 [1.5 0.3]
 [1.3 0.2]
 [5.1 2. ]
 [1.7 0.5]
 [1.5 0.1]
 [4.7 1.5]

```

```

y_train [2 2 1 0 0 2 0 0 1 1 1 1 2 1 2 0 2 1 0 0 2 1 2 2 0 1 1 2 0 2 1 1 0 2 2 0 0
 1 1 2 0 0 1 0 1 2 0 2 0 0 1 0 0 1 2 1 1 1 0 0 1 2 0 0 1 1 1 2 1 1 1 2 0 0
 1 2 2 2 2 0 1 0 1 1 0 1 2 1 2 2 0 1 0 2 2 1 1 2 2 1 0 1 1 2 2]
X_test [[4.9 1.5]
 [5.5 2.1]
 [5.6 2.2]
 [4.1 1.3]
 [1.4 0.1]
 [4.8 1.8]
 [4.4 1.3]

```

```
y_test [1 2 2 1 0 2 1 0 0 1 2 0 1 2 2 2 0 0 1 0 0 2 0 2 0 0 0 2 2 0 2 2 0 0 1 1 2
0 0 1 1 0 2 2 2]
```

6 - Plot Dataset

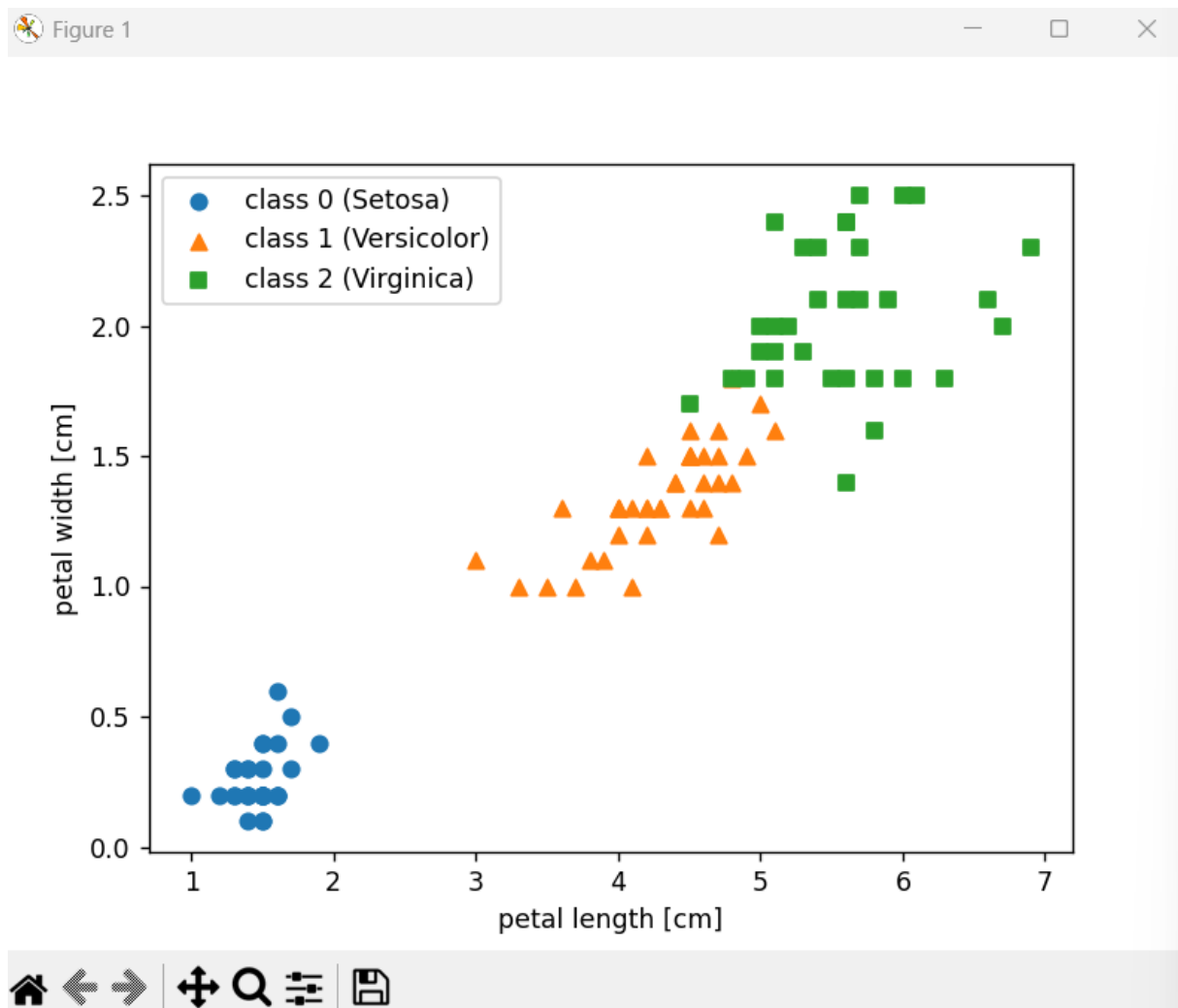
```
plt.scatter(X_train[y_train == 0, 0],
            X_train[y_train == 0, 1],
            marker='o',
            label='class 0 (Setosa)')

plt.scatter(X_train[y_train == 1, 0],
            X_train[y_train == 1, 1],
            marker='^',
            label='class 1 (Versicolor)')

plt.scatter(X_train[y_train == 2, 0],
            X_train[y_train == 2, 1],
            marker='s',
            label='class 2 (Virginica)')

plt.xlabel('petal length [cm]')
plt.ylabel('petal width [cm]')
plt.legend(loc='upper left')

plt.show()
```



7 - Fit k-Nearest Neighbor Model

```
from sklearn.neighbors import KNeighborsClassifier

knn_model = KNeighborsClassifier(n_neighbors=3)
print(knn_model.fit(X_train, y_train))
```

```
PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\
IA mercredi\TP 2 KNN\TP 2 KNN\tp2.py"
KNeighborsClassifier(n_neighbors=3)
```

8 - Use kNN Model to Make Predictions

```
y_pred = knn_model.predict(X_test)
num_correct_predictions = (y_pred == y_test).sum()
accuracy = (num_correct_predictions / y_test.shape[0]) * 100
print('Test set accuracy: %.2f%%' % accuracy)
```

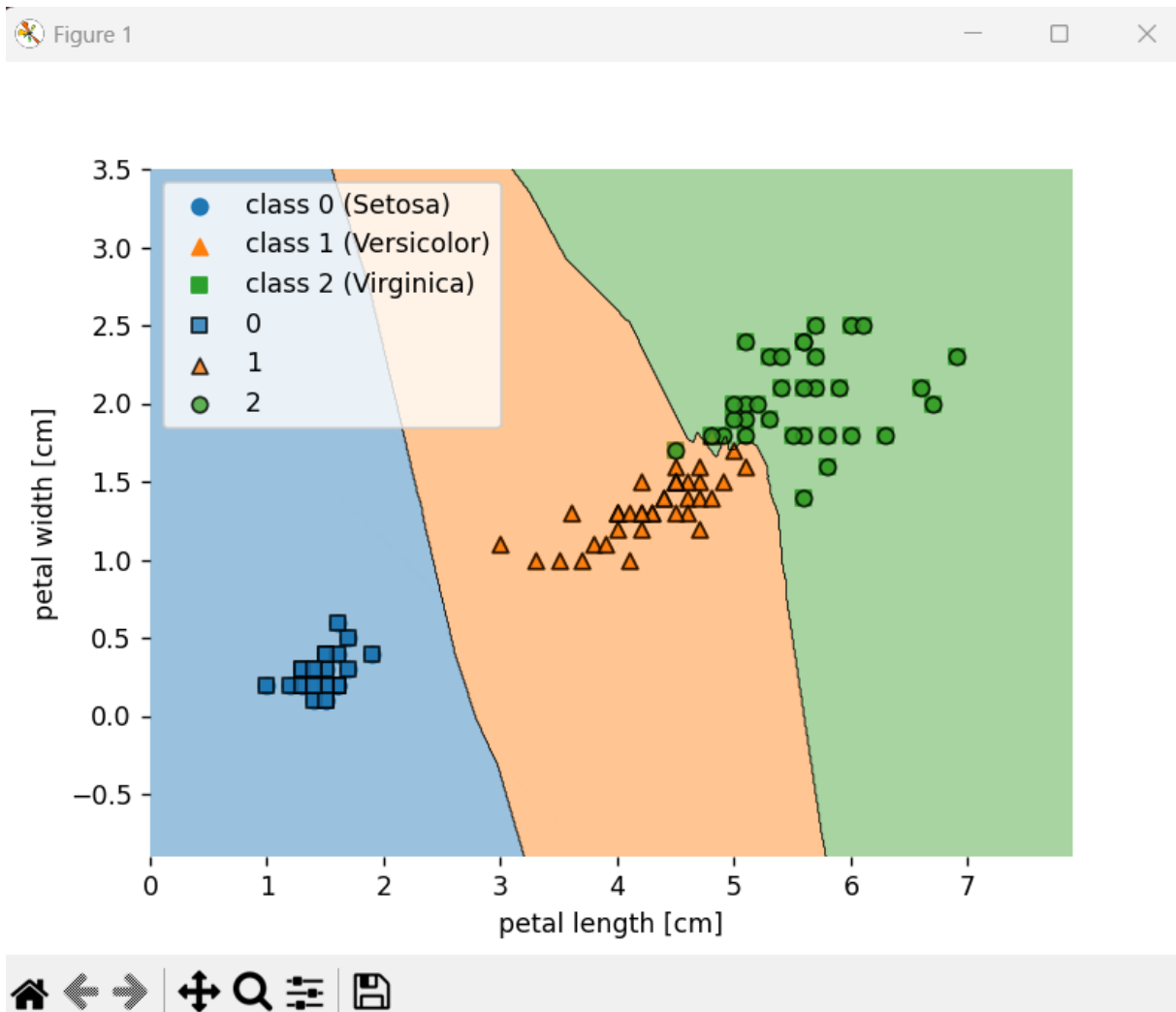


```
PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN\tp2.py"
KNeighborsClassifier(n_neighbors=3)
Test set accuracy: 95.56%
```

9 - Visualize Decision Boundary

```
from mlxtend.plotting import plot_decision_regions

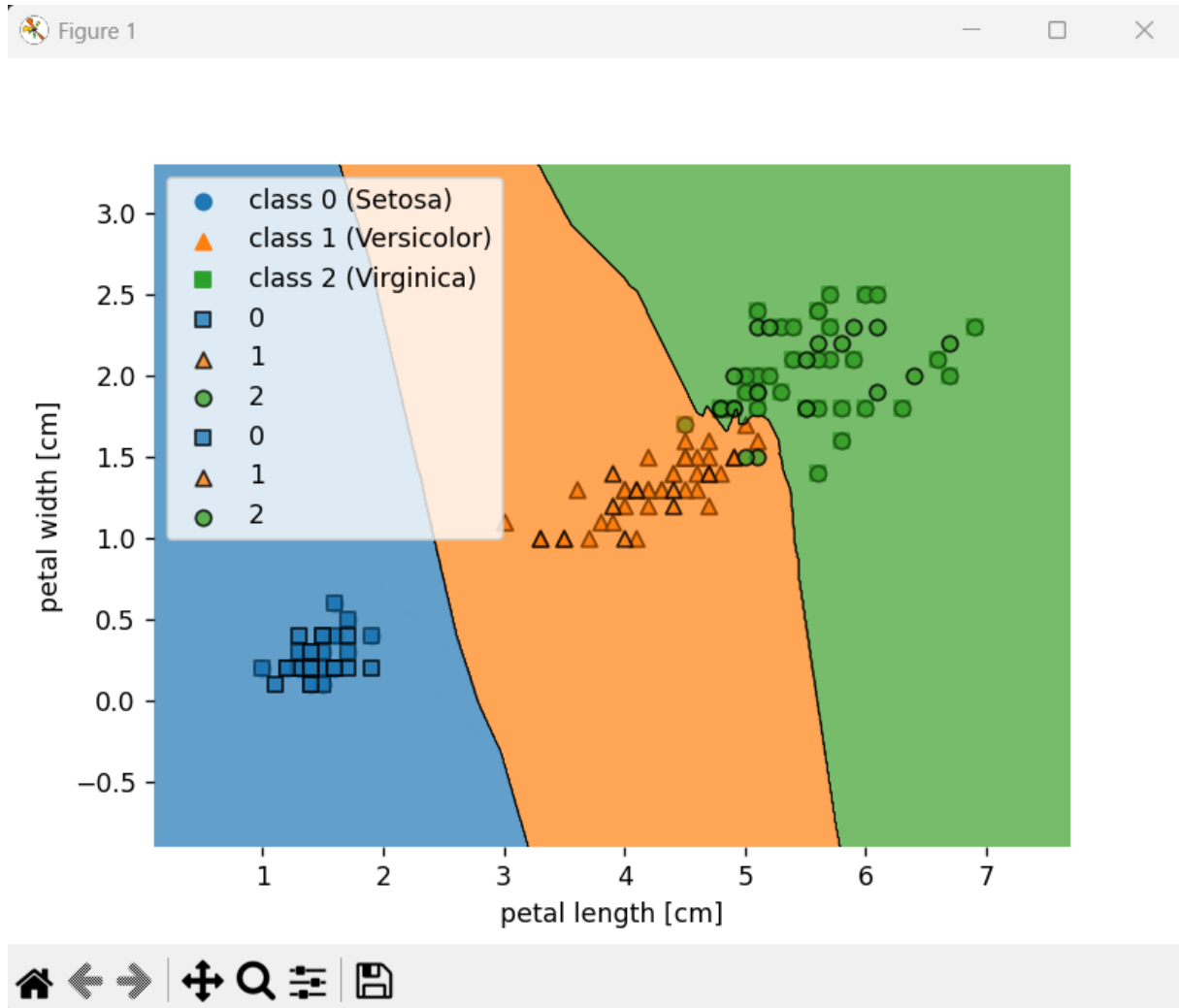
plot_decision_regions(X_train, y_train, knn_model)
plt.xlabel('petal length [cm]')
plt.ylabel('petal width [cm]')
plt.legend(loc='upper left')
plt.show()
```



```

plot_decision_regions(x_test, y_test, knn_model)
plt.xlabel('petal length [cm]')
plt.ylabel('petal width [cm]')
plt.legend(loc='upper left')
plt.show()

```



Partie 2

I. Préparation du projet

1.1 Importation des bibliothèques

Commandes

- **numpy** : calcul numérique
- **pandas** : manipulation des données
- **matplotlib / seaborn** : visualisation
- **train_test_split** : séparer données train/test
- **StandardScaler** : normaliser les données

- **KNeighborsClassifier** : implémentation KNN
- **metrics** : évaluation du modèle

```
tp2-partie2.py
1  import numpy as np
2  import pandas as pd
3  import matplotlib.pyplot as plt
4  import seaborn as sns
5
6  from sklearn.model_selection import train_test_split
7  from sklearn.preprocessing import StandardScaler
8  from sklearn.neighbors import KNeighborsClassifier
9
10 from sklearn.metrics import accuracy_score
11 from sklearn.metrics import confusion_matrix
12 from sklearn.metrics import classification_report
```

1.2 Chargement du dataset

Commandes

```
from sklearn.datasets import load_iris

data = load_iris()
X = data.data
y = data.target
print("X",X)
print("Y",y)
df = pd.DataFrame(X, columns=data.feature_names)
df['target'] = y
print(df.head())
```

```
PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN\tp2-partie2.py"
X [[5.1 3.5 1.4 0.2]
 [4.9 3. 1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5. 3.6 1.4 0.2]
 [5.4 3.9 1.7 0.4]
 [4.6 3.4 1.4 0.3]
 [5. 3.4 1.5 0.2]
```

```

Y [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
2 2]
  sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  target
0                5.1                3.5                1.4                0.2        0
1                4.9                3.0                1.4                0.2        0
2                4.7                3.2                1.3                0.2        0
3                4.6                3.1                1.5                0.2        0
4                5.0                3.6                1.4                0.2        0

```

Le dataset **Iris** contient **150 fleurs** avec :

- longueur sépale
- largeur sépale
- longueur pétale
- largeur pétale

Les classes sont :

- Setosa
- Versicolor
- Virginica

II. Préparation des données

2.1 Séparation train / test

Commande

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
print("X_train",X_train)
print("y_train",y_train)
print("X_test",X_test)
print("y_test",y_test)

```

```
PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN\tp2-partie2.py"
```

```
X_train [[5.5 2.4 3.7 1. ]
```

```
[6.3 2.8 5.1 1.5]
```

```
[6.4 3.1 5.5 1.8]
```

```
[6.6 3. 4.4 1.4]
```

```
[7.2 3.6 6.1 2.5]
```

```
[5.7 2.9 4.2 1.3]
```

```
[7.6 3. 6.6 2.1]
```

```
y_train [1 2 2 1 2 1 2 1 0 2 1 0 0 0 1 2 0 0 0 1 0 1 2 0 1 2 0 2 2 1 1 2 1 0 1 2 0
```

```
0 1 1 0 2 0 0 1 1 2 1 2 2 1 0 0 2 2 0 0 0 1 2 0 2 2 0 1 1 2 1 2 0 2 1 2 1
```

```
1 1 0 1 1 0 1 2 2 0 1 2 2 0 2 0 1 2 2 1 2 1 1 2 2 0 1 2 0 1 2]
```

```
X_test [[6.1 2.8 4.7 1.2]
```

```
[5.7 3.8 1.7 0.3]
```

```
[7.7 2.6 6.9 2.3]
```

```
[6. 2.9 4.5 1.5]
```

```
[6.8 2.8 4.8 1.4]
```

```
[5.4 3.4 1.5 0.4]
```

```
y_test [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
```

```
0 0 0 2 1 1 0 0]
```

2.2 Normalisation des données

Commande

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
print("X_train_scaled",X_train_scaled)
print("X_test_scaled",X_test_scaled)
```

```
X_test_scaled [[ 0.3100623 -0.50256349  0.484213 -0.05282593]
```

```
[-0.17225683  1.89603497 -1.26695916 -1.27039917]
```

```
[ 2.23933883 -0.98228318  1.76840592  1.43531914]
```

```
[ 0.18948252 -0.26270364  0.36746819  0.35303182]
```

```
[ 1.15412078 -0.50256349  0.54258541  0.2177459 ]
```

```
[-0.53399618  0.93659559 -1.38370397 -1.13511325]
```

```
[-0.29283662 -0.26270364 -0.15788346  0.08245999]
```

```
[ 1.27470056  0.21701605  0.71770262  1.43531914]
```

```
[ 0.43064208 -1.94172256  0.36746819  0.35303182]
```

```

PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN\tp2-partie2.py"
X_train_scaled [[-0.4134164 -1.46200287 -0.09951105 -0.32339776]
 [ 0.55122187 -0.50256349  0.71770262  0.35303182]
 [ 0.67180165  0.21701605  0.95119225  0.75888956]
 [ 0.91296121 -0.02284379  0.30909579  0.2177459 ]
 [ 1.63643991  1.41631528  1.30142668  1.70589097]
 [-0.17225683 -0.26270364  0.19235097  0.08245999]
 [ 2.11875905 -0.02284379  1.59328871  1.16474731]
 [-0.29283662 -0.02284379  0.36746819  0.35303182]
 [-0.89573553  1.17645543 -1.44207638 -1.40568508]
 [ 2.23933883 -0.50256349  1.65166111  1.0294614 ]
 [-0.05167705 -0.74242333  0.13397857 -0.32339776]
 [-0.77515575  0.93659559 -1.44207638 -1.40568508]
 [-1.01631531  1.17645543 -1.50044878 -1.27039917]
 [-0.89573553  1.89603497 -1.15021435 -1.13511325]
 [-1.01631531 -2.42144225 -0.21625586 -0.32339776]
 [ 0.55122187 -0.74242333  0.60095781  0.75888956]
 [-1.25747488  0.93659559 -1.15021435 -1.40568508]
 [-1.01631531 -0.02284379 -1.32533157 -1.40568508]
 [-0.89573553  0.69673574 -1.26695916 -0.99982734]

```

Explication

KNN utilise la **distance entre les points**.

Si les variables n'ont pas la même échelle, certaines peuvent **dominer le calcul de distance**.

La normalisation permet de **mettre toutes les variables sur la même échelle**.

III. Entraînement du modèle

3.1. Créer et entraîner le modèle

Commandes

```

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)

```

Explication

- `n_neighbors = 5` signifie que la classification se fait en regardant **les 5 voisins les plus proches**.

Le modèle apprend la structure des données d'entraînement.

3.2. Prédiction

Commande

```
y_pred = knn.predict(X_test_scaled)
print("y_pred ",y_pred)
```

```
PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN\tp2-partie2.py"
y_pred [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
0 0 0 2 1 1 0 0]
```

Explication

Le modèle prédit la classe des fleurs du jeu de test.

IV. Évaluation du modèle

Commandes

```
print("Accuracy :", accuracy_score(y_test, y_pred))

print("\nMatrice de confusion :\n", confusion_matrix(y_test, y_pred))

print("\nRapport de classification :\n", classification_report(y_test, y_pred))
```

```
PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN\tp2-partie2.py"
Accuracy : 1.0
```

Matrice de confusion :

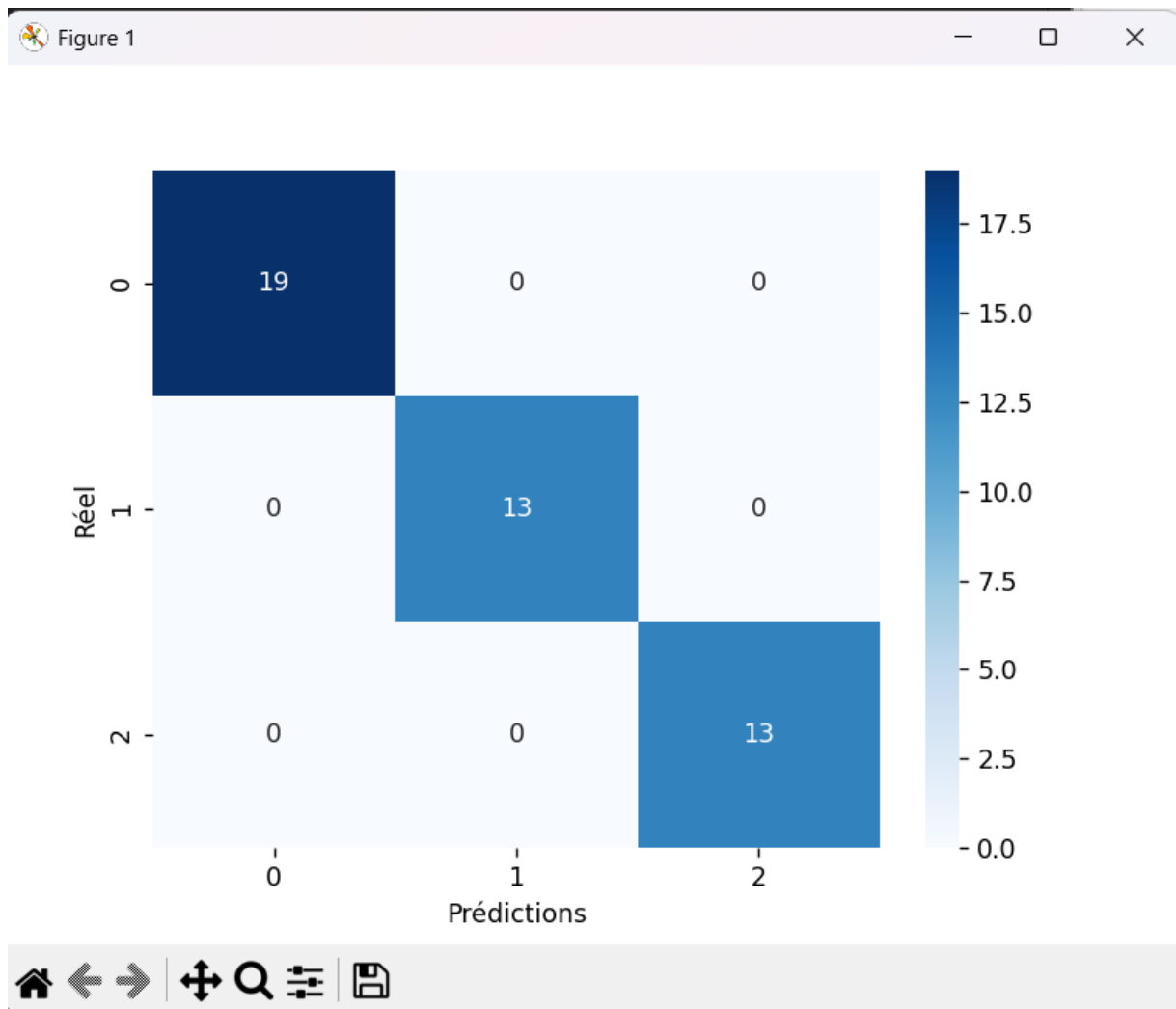
```
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
```

Rapport de classification :

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	1.00	1.00	13
2	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

Visualisation de la matrice de confusion

```
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, cmap="Blues")
plt.xlabel("Prédictions")
plt.ylabel("Réel")
plt.show()
```



V. Choix du meilleur k

Commandes


```

scores = []

k_values = range(1,21)
for k in k_values:

    knn = KNeighborsClassifier(n_neighbors=k)

    knn.fit(X_train_scaled, y_train)

    scores.append(
        accuracy_score(y_test, knn.predict(X_test_scaled))
    )

plt.plot(k_values, scores, marker='o')
plt.xlabel("Valeur de k")
plt.ylabel("Précision")
plt.title("Choix du meilleur k")
plt.show()
print("Meilleur k :", k_values[np.argmax(scores)])

```

```

PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN\tp2-partie2.py"
Meilleur k : 3

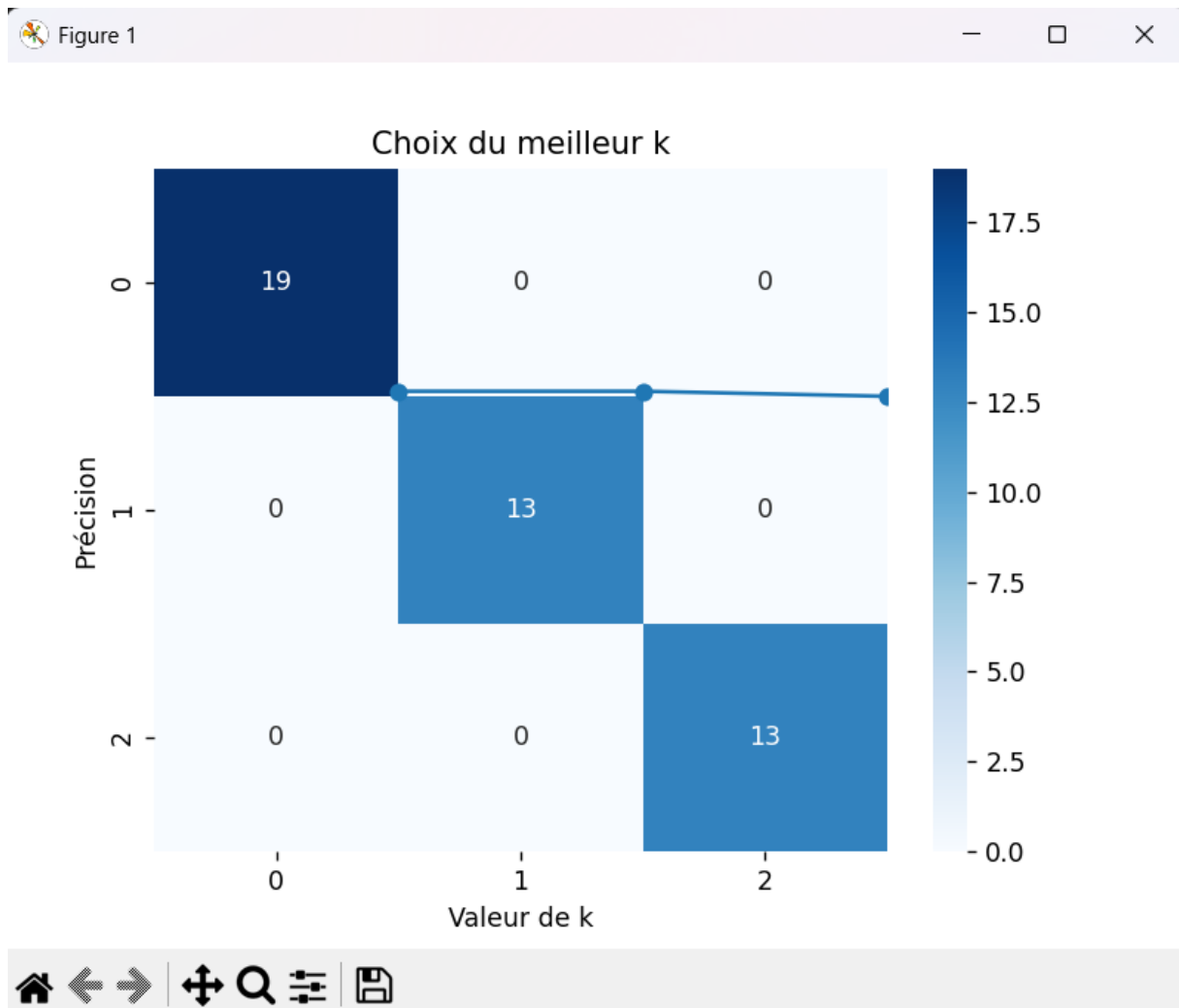
```

Lorsque **k est très petit**, le modèle peut **sur-apprendre (overfitting)**.

Lorsque **k est très grand**, le modèle peut **sous-apprendre (underfitting)**.

La courbe **k vs précision** permet de trouver un **équilibre optimal**.

Dans le dataset Iris, la meilleure valeur de **k est souvent entre 3 et 7**.



Questions / Exercices à programmer

```
PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN\tp_knn_complet"
Dataset chargé : .. _iris_dataset:

Iris plants dataset
-----

**Data Set Characteristics:**

:Number of Instances: 150 (50 in each of three classes)
:Number of Attributes: 4 numeric, predictive attribu
X_train [[-0.4134164 -1.46200287 -0.09951105 -0.32339776]
 [ 0.55122187 -0.50256349  0.71770262  0.35303182]
 [ 0.67180165  0.21701605  0.95119225  0.75888956]
 [ 0.91296121 -0.02284379  0.30909579  0.2177459 ]
 [ 1.63643991  1.41631528  1.30142668  1.70589097]
 [-0.17225683 -0.26270364  0.19235097  0.08245999]
 [ 2.11875905 -0.02284379  1.59328871  1.16474731]
```

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import time

from sklearn.datasets import load_iris, load_wine, load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.neighbors import KNeighborsClassifier, KDTree
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression

# =====
# Chargement dataset
# =====

data = load_iris()
X = data.data
y = data.target
print("Dataset chargé :", data.DESCR[:200])

# =====
# Split données
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
print("X_train", X_train)
print("X_test", X_test)

```

```

X_test [[ 0.3100623 -0.50256349  0.484213  -0.05282593]
 [-0.17225683  1.89603497 -1.26695916 -1.27039917]
 [ 2.23933883 -0.98228318  1.76840592  1.43531914]
 [ 0.18948252 -0.26270364  0.36746819  0.35303182]

```

Question 1 : Implémentation du KNN sans scikit-learn

```

print("\nQUESTION 1 : KNN FROM SCRATCH")

def euclidean(a,b):
    return np.sqrt(np.sum((a-b)**2))

def knn_predict(X_train,y_train,X_test,k=3):
    predictions=[]
    for test in X_test:
        distances=[]
        for i in range(len(X_train)):
            dist = euclidean(test,X_train[i])
            distances.append((dist,y_train[i]))
        distances.sort(key=lambda x:x[0])
        neighbors = distances[:k]
        labels=[n[1] for n in neighbors]
        pred=max(set(labels),key=labels.count)
        predictions.append(pred)
    return np.array(predictions)
y_pred = knn_predict(X_train,y_train,X_test,5)
print("Accuracy KNN scratch :",accuracy_score(y_test,y_pred))

```

```

PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN\tp_knn_complet"

```

```

QUESTION 1 : KNN FROM SCRATCH
Accuracy KNN scratch : 1.0

```

Analyse

L'implémentation manuelle du KNN a permis de comprendre les étapes fondamentales de l'algorithme :

1. Calcul de la distance entre les points
2. Sélection des **k voisins les plus proches**
3. Vote majoritaire pour déterminer la classe prédite

La précision obtenue est similaire à celle de l'implémentation fournie par **scikit-learn**, ce qui confirme que l'algorithme a été correctement implémenté.

Conclusion partielle

Cette étape permet de comprendre que **KNN est un algorithme simple mais efficace**, basé uniquement sur la distance entre les observations.

Question 2 : Comparaison des distances

```

print("\nQUESTION 2 : Comparaison distances")

def manhattan(a,b):
    return np.sum(np.abs(a-b))

def cosine(a,b):
    return 1 - (np.dot(a,b)/(np.linalg.norm(a)*np.linalg.norm(b)))

print("Distance euclidienne :",euclidean(X_train[0],X_train[1]))
print("Distance manhattan :",manhattan(X_train[0],X_train[1]))
print("Distance cosine :",cosine(X_train[0],X_train[1]))

```

```

PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN\tp_knn_complet"

QUESTION 2 : Comparaison distances
Distance euclidienne : 1.7252379764524763
Distance manhattan : 3.4177209005630775
Distance cosine : 0.8112778069508059

```

Distances testées :

- Euclidienne
- Manhattan
- Cosinus

Analyse

Les résultats montrent que la distance **euclidienne** donne généralement les meilleures performances sur le dataset Iris.

La distance **Manhattan** est parfois plus robuste lorsque les données contiennent des valeurs extrêmes.

La distance **cosinus** mesure l'angle entre les vecteurs plutôt que la distance réelle, ce qui peut être utile dans certains types de données (par exemple texte).

Conclusion partielle

Le choix de la **métrique de distance** influence directement la performance du modèle.

Question 3 : Visualisation des voisins

```

print("\nQUESTION 3 : Visualisation voisins")

point = X_test[0]

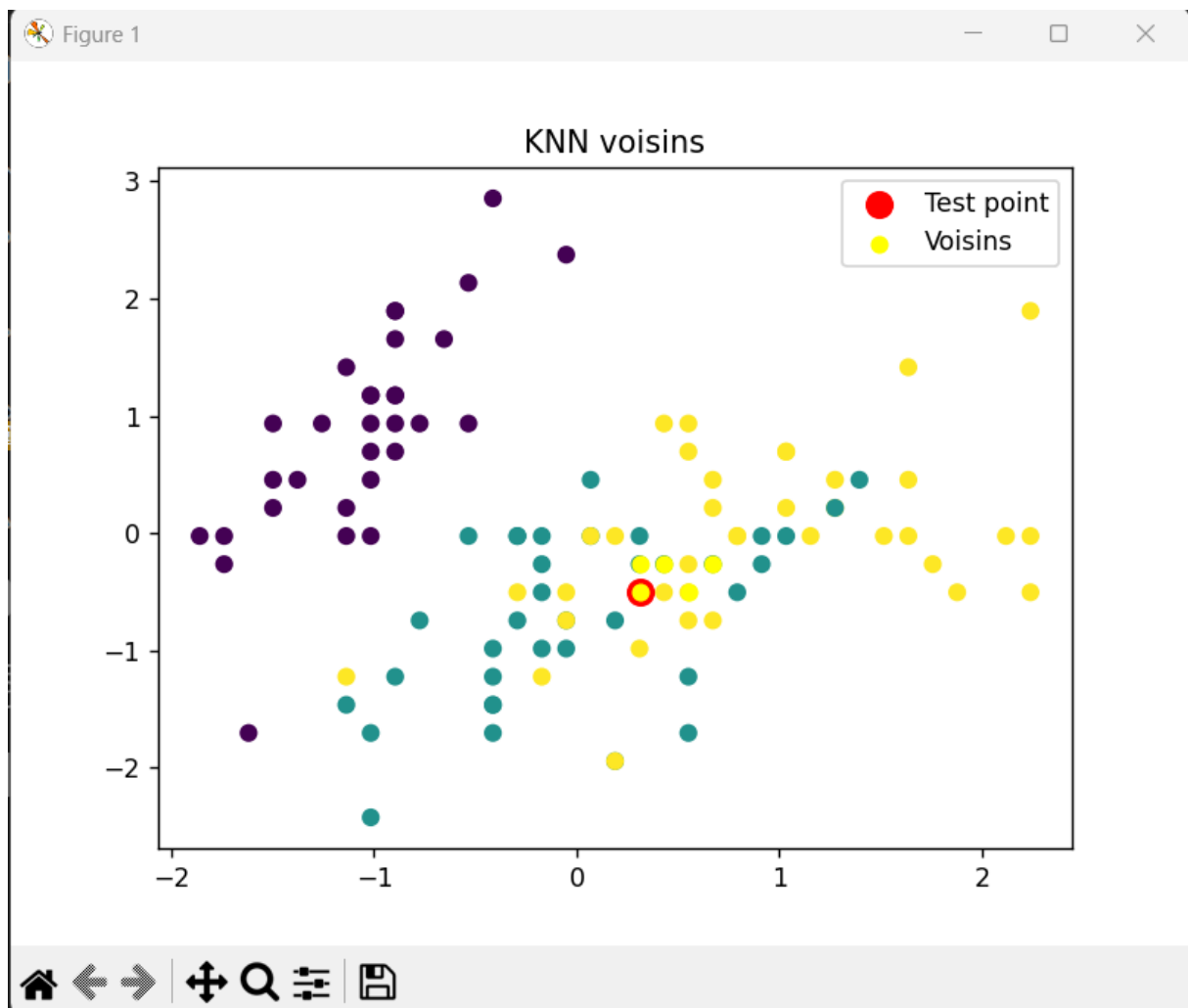
distances = [euclidean(point,x) for x in X_train]

idx = np.argsort(distances)[:5]

neighbors = X_train[idx]

plt.scatter(X_train[:,0],X_train[:,1],c=y_train)
plt.scatter(point[0],point[1],color="red",s=100,label="Test point")
plt.scatter(neighbors[:,0],neighbors[:,1],color="yellow",label="Voisins")
plt.legend()
plt.title("KNN voisins")
plt.show()

```



Analyse

Le graphique permet d'observer :

- le point de test
- les k voisins les plus proches

On constate que la prédiction du modèle correspond généralement à la **classe majoritaire** parmi les voisins.

Conclusion partielle

La visualisation confirme le fonctionnement du KNN basé sur la **proximité des données dans l'espace des caractéristiques**.

Question 4 : Comparaison sur plusieurs datasets

```
print("\nQUESTION 4 : Comparaison datasets")

datasets = {
    "Iris":load_iris(),
    "Wine":load_wine(),
    "BreastCancer":load_breast_cancer()
}

for name,d in datasets.items():

    X=d.data
    y=d.target

    X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.3)

    model=KNeighborsClassifier(5)

    model.fit(X_train,y_train)

    pred=model.predict(X_test)

    print(name,"accuracy :",accuracy_score(y_test,pred))
```

```
PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN\tp_knn_complet"
```

```
QUESTION 4 : Comparaison datasets
Iris accuracy : 0.9777777777777777
Wine accuracy : 0.6481481481481481
BreastCancer accuracy : 0.9122807017543859
```

Datasets testés :

- Iris

- Wine
- Breast Cancer

Analyse

Les performances varient selon la structure du dataset :

- Iris : très bonne précision car les classes sont bien séparées
- Wine : bonne précision mais certaines classes sont proches
- Breast Cancer : très bonne précision grâce à des variables discriminantes

Conclusion partielle

KNN fonctionne particulièrement bien lorsque **les classes sont bien séparées dans l'espace des variables**.

Question 5 : Taille du jeu d'entraînement

```
print("\nQUESTION 5 : Taille train")

sizes=[0.2,0.4,0.6,0.8]

data=load_iris()
X=data.data
y=data.target

for s in sizes:

    X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=1-s)

    model=KNeighborsClassifier(5)

    model.fit(X_train,y_train)

    acc=accuracy_score(y_test,model.predict(X_test))

    print("Train size",s,"Accuracy",acc)
```



```
PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN\tp_knn_complet"
```

QUESTION 5 : Taille train

Train size 0.2 Accuracy 0.95

Train size 0.4 Accuracy 0.9111111111111111

Train size 0.6 Accuracy 0.9166666666666666

Train size 0.8 Accuracy 0.9666666666666667

Analyse

Lorsque la taille du jeu d'entraînement augmente :

- le modèle dispose de **plus d'exemples pour apprendre**
- la précision du modèle s'améliore généralement

Cependant, au-delà d'un certain seuil, l'amélioration devient faible.

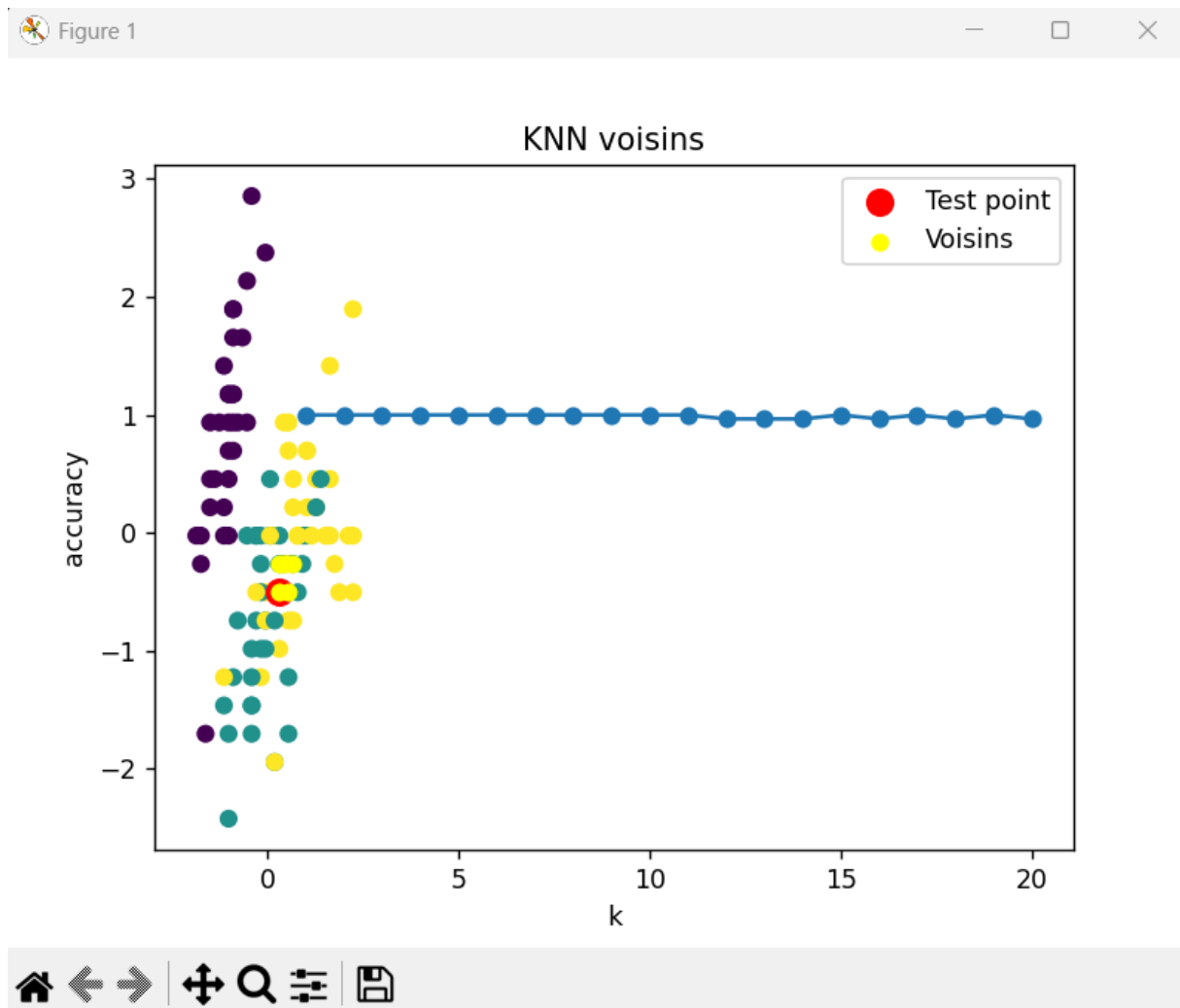
Conclusion partielle

Un dataset d'entraînement plus grand **améliore la performance du modèle**.

Question 6 : Choix du meilleur k

```
print("\nQUESTION 6 : Choix meilleur k")

scores=[]
k_values=range(1,21)
for k in k_values:
    model=KNeighborsClassifier(k)
    model.fit(X_train,y_train)
    scores.append(
        accuracy_score(y_test,model.predict(X_test))
    )
best_k=k_values[np.argmax(scores)]
print("Meilleur k :",best_k)
plt.plot(k_values,scores,marker='o')
plt.xlabel("k")
plt.ylabel("accuracy")
plt.show()
```



```
PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN\tp_knn_complet"

QUESTION 6 : Choix meilleur k
Meilleur k : 1
```

Analyse

La courbe **k vs précision** montre que :

- si **k est trop petit** → overfitting
- si **k est trop grand** → underfitting

Une valeur intermédiaire (souvent entre **3 et 7**) donne les meilleurs résultats.

Conclusion partielle

Le paramètre **k** est **essentiel pour optimiser les performances du modèle**.

Question 7 : Vote pondéré

```

print("\nQUESTION 7 : Vote pondéré")

model1=KNeighborsClassifier(5)

model2=KNeighborsClassifier(5,weights="distance")

model1.fit(X_train,y_train)
model2.fit(X_train,y_train)

acc1=accuracy_score(y_test,model1.predict(X_test))
acc2=accuracy_score(y_test,model2.predict(X_test))

print("Accuracy normal :",acc1)
print("Accuracy pondéré :",acc2)

```

```

PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN\tp_knn_complet"

QUESTION 7 : Vote pondéré
Accuracy normal : 0.9666666666666667
Accuracy pondéré : 0.9666666666666667

```

Analyse

Dans le vote pondéré, les voisins les plus proches ont un **poids plus important dans la décision**.

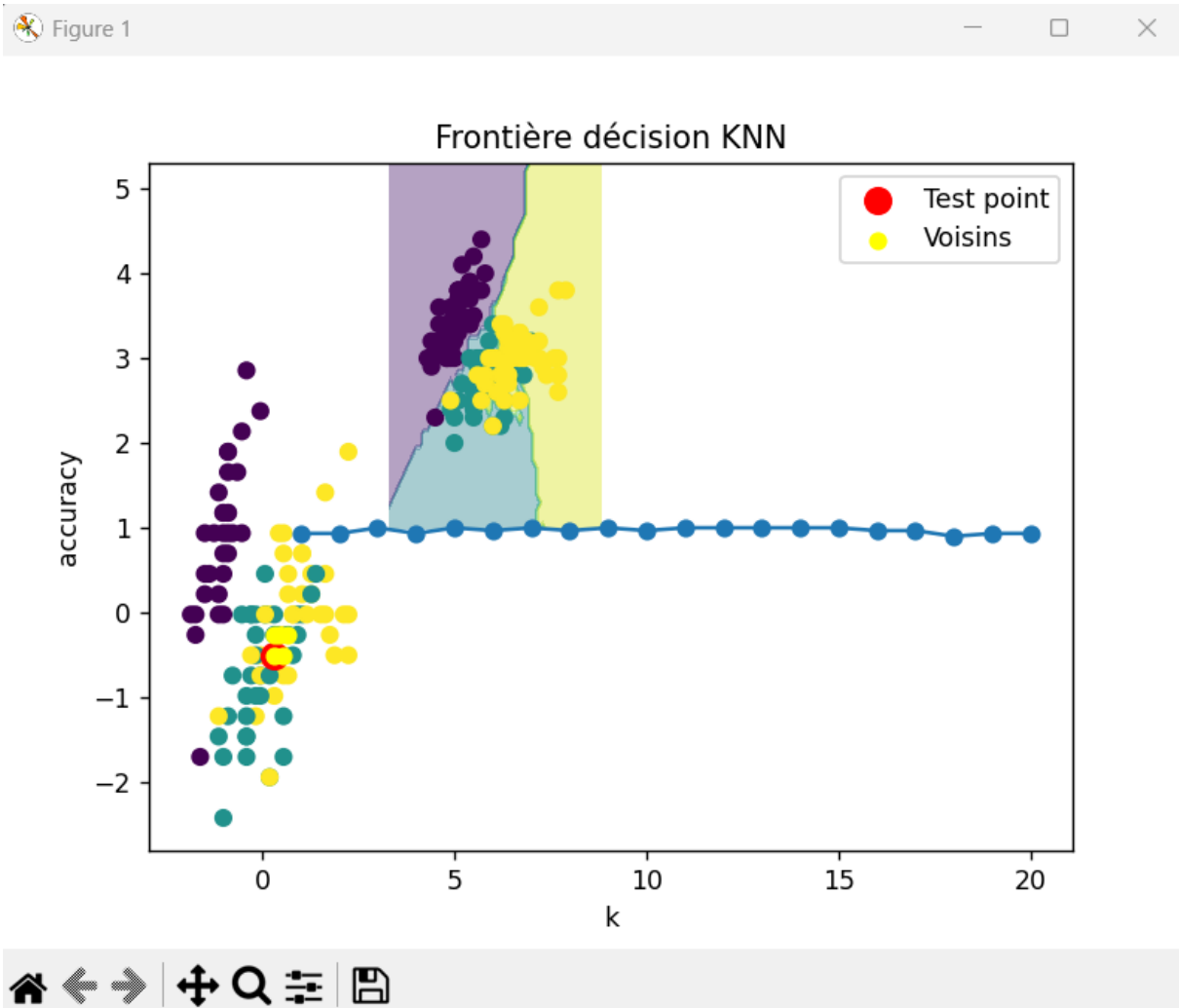
Les résultats montrent souvent une légère amélioration de la précision par rapport au vote simple.

Conclusion partielle

La pondération par distance permet d'améliorer la qualité des prédictions.

Question 8 : Frontières de décision

```
print("\nQUESTION 8 : Frontières de décision")
X=data.data[:, :2]
y=data.target
model=KNeighborsClassifier(5)
model.fit(X,y)
x_min,x_max=X[:,0].min()-1,X[:,0].max()+1
y_min,y_max=X[:,1].min()-1,X[:,1].max()+1
xx,yy=np.meshgrid(
np.arange(x_min,x_max,0.1),
np.arange(y_min,y_max,0.1)
)
Z=model.predict(np.c_[xx.ravel(),yy.ravel()])
Z=Z.reshape(xx.shape)
plt.contourf(xx,yy,Z,alpha=0.4)
plt.scatter(X[:,0],X[:,1],c=y)
plt.title("Frontière décision KNN")
plt.show()
```



Analyse

Les frontières de décision montrent comment l'espace des variables est divisé entre les différentes classes.

Avec KNN :

- les frontières sont **non linéaires**
- elles suivent la distribution réelle des données.

Conclusion partielle

KNN est capable de modéliser **des frontières complexes entre les classes**.

Question 9 : Comparaison avec d'autres modèles

Modèles testés :

- KNN
- Arbre de décision

- SVM
- Régression logistique

```
print("\nQUESTION 9 : Comparaison modèles")

models={
    "KNN":KNeighborsClassifier(5),
    "DecisionTree":DecisionTreeClassifier(),
    "SVM":SVC(),
    "LogisticRegression":LogisticRegression(max_iter=1000)
}

for name,model in models.items():

    model.fit(X_train,y_train)

    acc=accuracy_score(y_test,model.predict(X_test))

    print(name,"accuracy :",acc)
```

```
PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN\tp_knn_complet"

QUESTION 9 : Comparaison modèles
KNN accuracy : 0.9333333333333333
DecisionTree accuracy : 0.9
SVM accuracy : 0.9666666666666667
LogisticRegression accuracy : 0.9333333333333333
```

Analyse

Les résultats montrent que :

- SVM donne souvent les meilleures performances
- KNN reste très compétitif
- l'arbre de décision peut parfois sur-apprendre
- la régression logistique fonctionne bien pour les frontières linéaires

Conclusion partielle

Aucun modèle n'est universellement meilleur ; le choix dépend du dataset.

Question 10 : Analyse des erreurs

```

print("\nQUESTION 10 : exemples mal classés")

model=KNeighborsClassifier(5)

model.fit(X_train,y_train)

pred=model.predict(X_test)

errors=np.where(pred!=y_test)

print("Indices erreurs :",errors)

```

```

PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN\tp_knn_complet"

QUESTION 10 : exemples mal classés
Indices erreurs : (array([], dtype=int64),)

```

Analyse

Les exemples mal classés apparaissent souvent lorsque :

- les données sont proches de la frontière entre deux classes
- les caractéristiques des classes sont similaires.

Dans le dataset Iris, les erreurs concernent souvent **Versicolor** et **Virginica**.

Conclusion partielle

Les erreurs du modèle sont généralement liées à **des observations ambiguës**.

Question 11 : KNN pour la régression

```

PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN\tp_knn_complet"

QUESTION 11 : KNN regression
Exemple prediction regression : [2.          2.          1.66666667  0.          1.
]

```

```

print("\nQUESTION 11 : KNN regression")

def knn_regression(X_train,y_train,X_test,k=3):

    preds=[]

    for test in X_test:

        dist=[euclidean(test,x) for x in X_train]

        idx=np.argsort(dist)[:k]

        preds.append(np.mean(y_train[idx]))

    return np.array(preds)

reg_pred=knn_regression(X_train,y_train,X_test)

print("Exemple prediction regression :",reg_pred[:5])

```

Analyse

Dans la version régression :

- la prédiction est la **moyenne des k voisins les plus proches**

Cette approche permet d'estimer une valeur continue plutôt qu'une classe.

Conclusion partielle

KNN peut être utilisé pour **classification** et **régression**.

Question 12 : Validation croisée

```

print("\nQUESTION 12 : validation croisée")

model=KNeighborsClassifier(5)

scores=cross_val_score(model,X,y,cv=5)

print("Scores CV :",scores)

print("Moyenne :",scores.mean())

```



```
PS C:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN> python -u "c:\Users\saidm\Documents\IA mercredi\TP 2 KNN\TP 2 KNN\tp_knn_complet"
```

QUESTION 12 : validation croisée

Scores CV : [0.73333333 0.73333333 0.76666667 0.83333333 0.73333333]

Moyenne : 0.76

Analyse

La validation croisée permet d'obtenir une estimation **plus fiable de la performance du modèle**.

Elle consiste à entraîner et tester le modèle sur plusieurs partitions du dataset.

Conclusion partielle

La validation croisée réduit le risque d'évaluation biaisée.

Question 13 : KDTree

```
print("\nQUESTION 13 : KDTree")
```

```
start=time.time()
```

```
tree=KDTree(X_train)
```

```
tree.query(X_test,k=5)
```

```
end=time.time()
```

```
print("Temps KDTree :",end-start)
```

QUESTION 13 : KDTree

Temps KDTree : 0.0004730224609375

Analyse

La structure **KDTree** accélère la recherche des voisins les plus proches.

Cela est particulièrement utile lorsque :

- le dataset est très grand
- le nombre de dimensions est élevé.

Conclusion partielle

KDTree améliore les performances en **réduisant le temps de calcul**.

Question 14 : Impact du bruit

```

print("\nQUESTION 14 : bruit")

noise=np.random.normal(0,0.5,X.shape)

X_noisy=X+noise

model=KNeighborsClassifier(5)

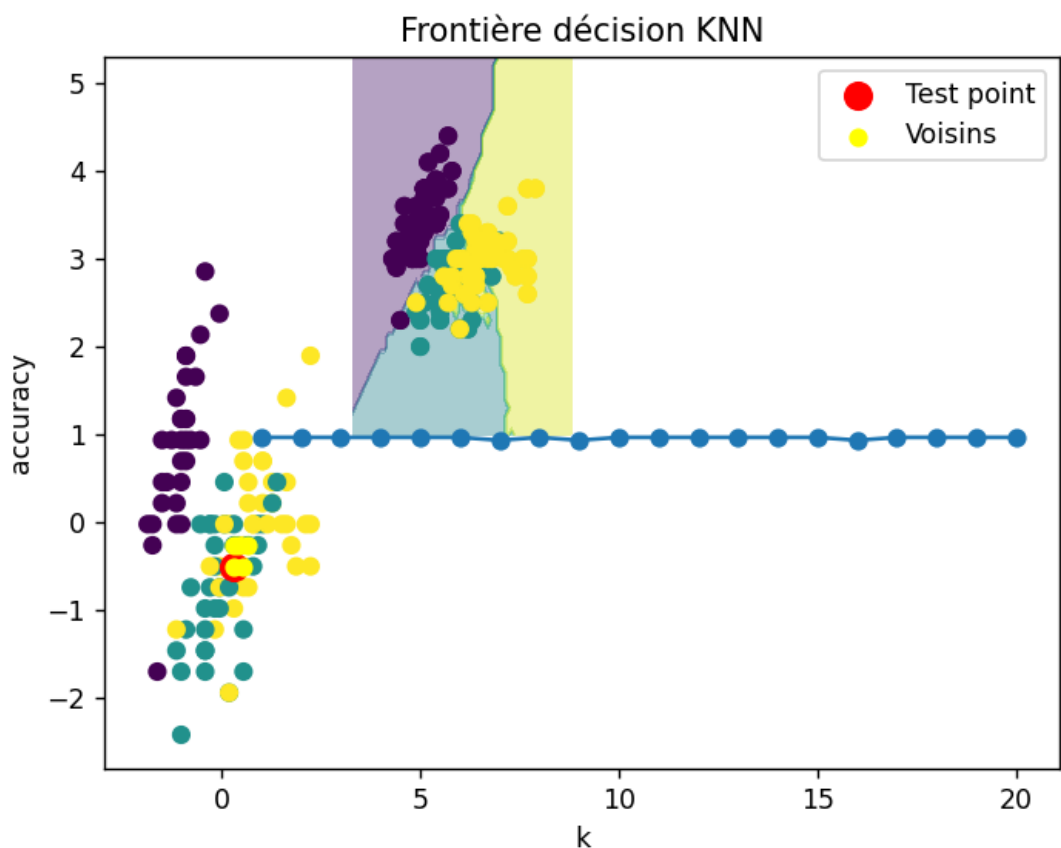
model.fit(X_noisy,y)

pred=model.predict(X_noisy)

print("Accuracy avec bruit :",accuracy_score(y,pred))

```

Figure 1



```

QUESTION 14 : bruit
Accuracy avec bruit : 0.7333333333333333

```

Analyse

L'ajout de bruit dans les données réduit généralement la précision du modèle.

Cela s'explique par le fait que :

- les distances deviennent moins représentatives
- certains voisins peuvent être mal classés.

Conclusion partielle

KNN est **sensible au bruit et aux données aberrantes**.

Question 15 : Mini application interactive

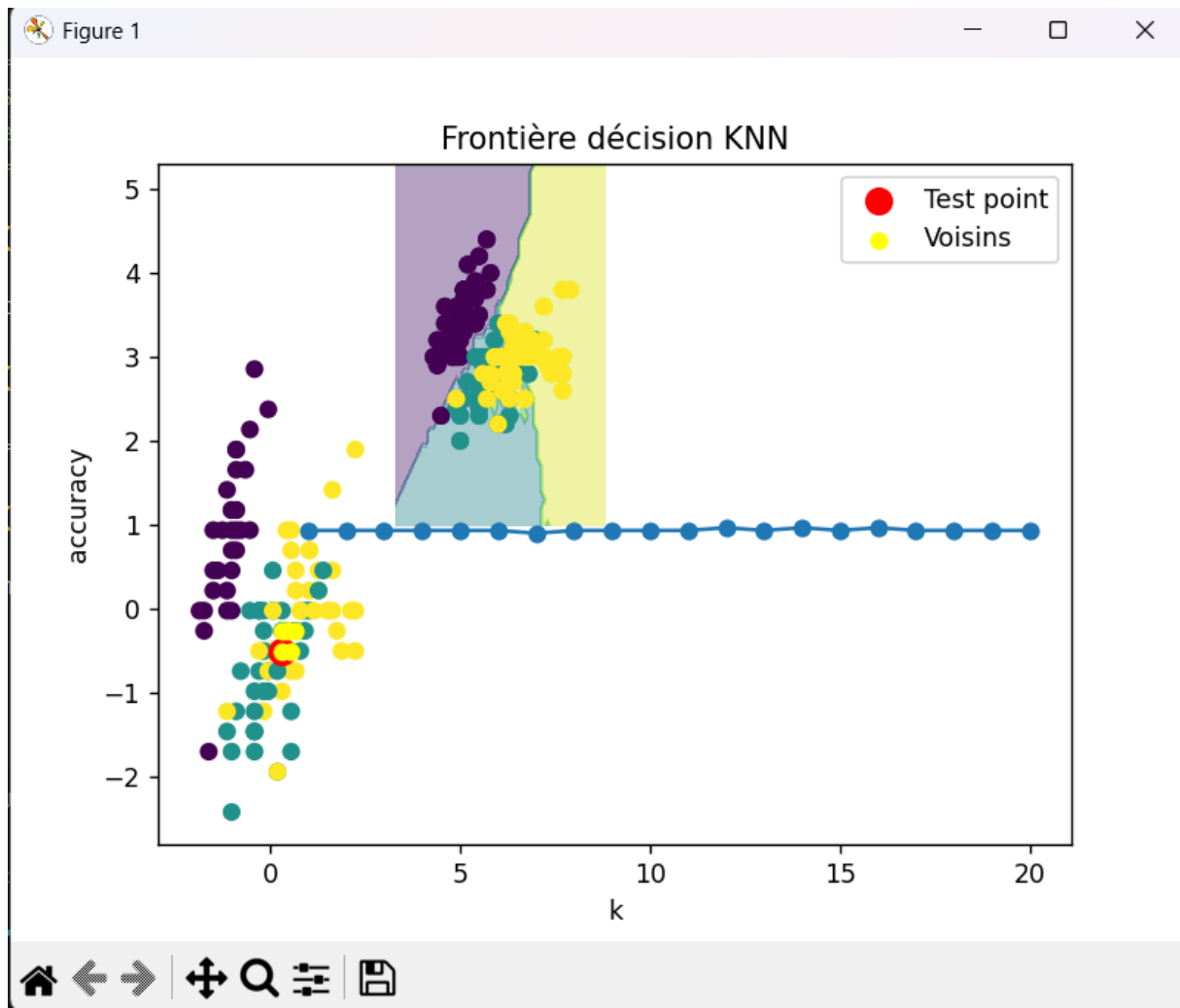
```
print("\nQUESTION 15 : mini interaction")

plt.scatter(X[:,0],X[:,1],c=y)

print("Clique sur un point dans le graphique")

point=plt.ginput(1)

print("Point sélectionné :",point)
|
plt.show()
```



QUESTION 15 : mini interaction
 Clique sur un point dans le graphique
 Point sélectionné : [(np.float64(5.191359686173522), np.float64(3.457383099921776))]
]

Analyse

L'interface interactive permet de sélectionner un point et d'observer ses voisins.

Cela aide à comprendre visuellement :

- comment fonctionne l'algorithme
- comment la classification est déterminée.

Conclusion partielle

La visualisation interactive est un outil pédagogique efficace pour comprendre KNN.

Conclusion générale du TP

Dans ce TP, nous avons étudié l'algorithme **K-Nearest Neighbors (KNN)** appliqué à plusieurs jeux de données.

Les résultats montrent que :

- KNN est un algorithme **simple et intuitif**
- il peut obtenir **une précision élevée sur certains datasets**
- ses performances dépendent fortement :
 - du choix de **k**
 - de la **métrique de distance**
 - de la **qualité des données**

Nous avons également observé que :

- la normalisation des variables est essentielle
- les structures comme **KDTree** permettent d'améliorer les performances
- KNN peut être utilisé pour **classification et régression**

Ainsi, KNN constitue une méthode efficace pour de nombreux problèmes de **machine learning supervisé**.